CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

CSPIT — Department of Computer Engineering · ITUE203 Web Development Frameworks

Student Name	Dhruv Bhoi	**Student ID**	**D26CE154**
Academic Year	2026–27 (ODD)	**Semester**	3rd Semester, B.Tech CE

Practical 13: PHP JSON API and AJAX/Fetch-Based CRUD without Page Reload

Practical Aim & Tools

Aim: Create a RESTful PHP JSON API endpoint (`api/events.php`) supporting GET and POST, and integrate AJAX/Fetch CRUD operations without page reloads, including loading spinners and toast notifications.

Tools & Technologies: PHP REST API, JSON Serialization, AJAX, Fetch API, DOM Replacement

1. RESTful JSON API Architecture (`api/events.php`)

StudentHub implements a dedicated headless API endpoint adhering to REST conventions:

• Content Negotiation: Emits `Content-Type: application/json; charset=utf-8` on every response.

• HTTP Verbs & Routing: GET requests query events with optional debounced search filters; POST requests handle authenticated CRUD actions (create, delete, toggle status).

• Standard HTTP Status Codes: Returns `200 OK` on success, `401 Unauthorized` for guests, `403 Forbidden` for invalid CSRF tokens, `422 Unprocessable Entity` for validation errors, and `500 Internal Error` for failures.

• Client-Side Controller (`assets/js/api-crud.js`): Intercepts form submissions, transmits JSON payloads with `X-CSRF-Token` headers, and smoothly updates the DOM table without full page refreshes.

2. Implementation Code: API Router & AJAX Client

Listing: api/events.php — Parameterized REST API with HTTP Status Emission

```
// api/events.php - JSON API Router
header('Content-Type: application/json; charset=utf-8');
require_once __DIR__ . '/../includes/functions.php';
require_once __DIR__ . '/../includes/db.php';
require_login(); // Requires authenticated session (401 response on failure)

if ($_SERVER['REQUEST_METHOD'] === 'GET') {
    $q = clean($_GET['q'] ?? '');
    $sql = 'SELECT id, title, category, event_date, venue, status FROM events';
    if ($q !== '') $sql .= ' WHERE title LIKE ? OR venue LIKE ?';
    $st = $pdo->prepare($sql);
```

```
    $q !== '' ? $st->execute(["%$q%", "%$q%"]) : $st->execute();
    echo json_encode(['success' => true, 'data' => $st->fetchAll()]);
    exit;
}
```

3. Verification Screenshot: Live AJAX Events Interface

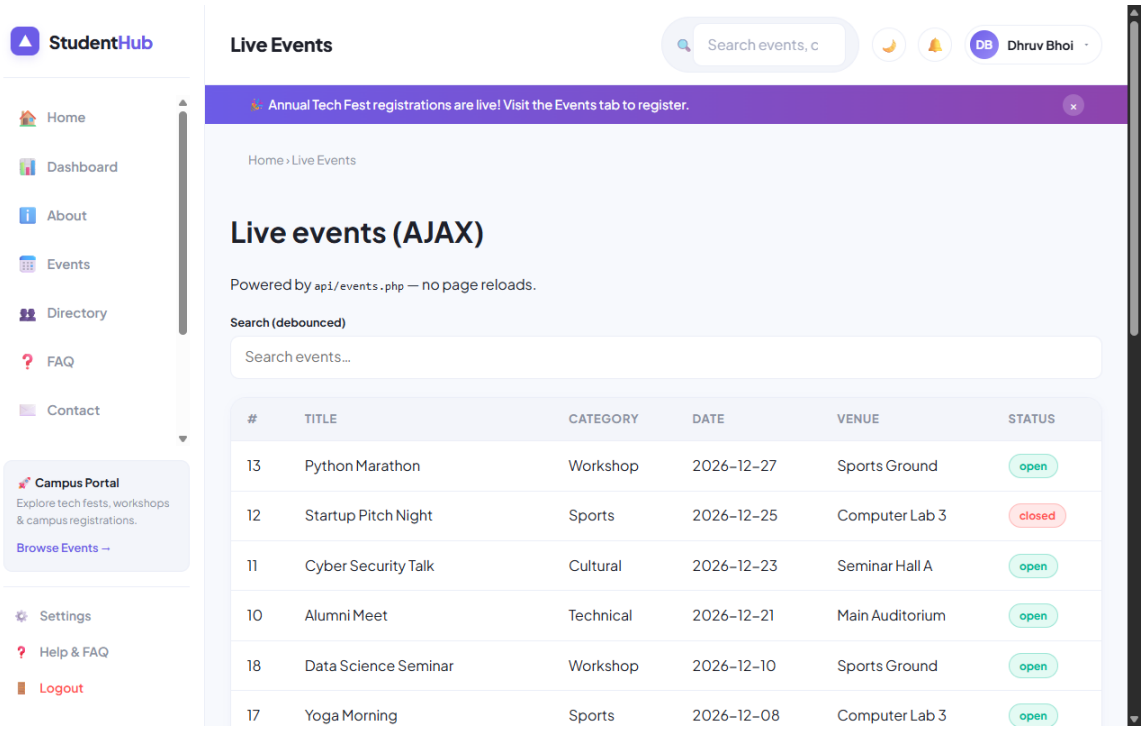


Figure: Live AJAX Events Dashboard (api-demo.php) Rendering Real-Time Updates without Reloads

4. Key Questions & Analysis

Q: How are CSRF tokens transmitted during asynchronous AJAX requests?

Ans: During page generation, the server places a CSRF token in a data attribute (`data-csrf="..."`). The JavaScript controller attaches this token to the custom request header `X-CSRF-Token`, which PHP validates via `csrf_check($_SERVER['HTTP_X_CSRF_TOKEN'])`.

Q: What are the benefits of Single Page Application (SPA) AJAX updates over traditional server-rendered pages?

Ans: AJAX updates significantly reduce network bandwidth by exchanging lightweight JSON rather than full HTML documents. The user experience is seamless without screen flashing, form inputs preserve focus, and responsiveness feels like a native desktop app.

Practical 13 Conclusion & Learning Outcomes

Constructed a RESTful PHP JSON API endpoint (api/events.php) and built a Single-Page Application (SPA) client interface (api-demo.php). Integrated asynchronous Fetch CRUD operations, custom X-CSRF-Token headers, and dynamic DOM rendering without full page reloads.